

Лабораторная работа №6

Мандатное разграничение прав в Linux

Павлова Татьяна Юрьевна

Содержание

1	Цель работы	5
2	Теоретическое введение	6
3	Выполнение лабораторной работы	8
4	Выводы	19
	Список литературы	20

Список иллюстраций

3.1	проверка режима работы SELinux	8
3.2	Проверка работы Apache	9
3.3	Контекст безопасности Apache	9
3.4	Состояние переключателей SELinux	10
3.5	Статистика по политике	10
3.6	Типы поддиректорий	11
3.7	Типы файлов	11
3.8	Создание файла	11
3.9	Контекст файла	11
3.10	Отображение файла	12
3.11	Изучение справки по команде	13
3.12	Изменение контекста	13
3.13	Отображение файла	14
3.14	Попытка прочесть лог-файл	14
3.15	Изменение файла	15
3.16	Изменение порта	16
3.17	Попытка прослушивания другого порта	16
3.18	Перезапуск сервера	17
3.19	Проверка сервера	17
3.20	Проверка порта 81	18
3.21	Удаление файла	18

Список таблиц

1 Цель работы

Целью данной работы является развитие навыков администрирования ОС Linux, получение первого практического знакомства с технологией SELinux, а также проверка работы на практике совместно с веб-сервером Apache.

2 Теоретическое введение

1. **SELinux (Security-Enhanced Linux)** обеспечивает усиление защиты путем внесения изменений как на уровне ядра, так и на уровне пространства пользователя, что превращает ее в действительно «непробиваемую» операционную систему. Впервые эта система появилась в четвертой версии CentOS, а в 5 и 6 версии реализация была существенно дополнена и улучшена.

SELinux имеет три основных режим работы:

- **Enforcing:** режим по умолчанию. При выборе этого режима все действия, которые каким-то образом нарушают текущую политику безопасности, будут блокироваться, а попытка нарушения будет зафиксирована в журнале.
- **Permissive:** в случае использования этого режима, информация о всех действиях, которые нарушают текущую политику безопасности, будут зафиксированы в журнале, но сами действия не будут заблокированы.
- **Disabled:** полное отключение системы принудительного контроля доступа.

Политика SELinux определяет доступ пользователей к ролям, доступ ролей к доменам и доступ доменов к типам. Контекст безопасности — все атрибуты SELinux — роли, типы и домены. Более подробно см. в [f].

2. **Apache** — это свободное программное обеспечение, с помощью которого можно создать веб-сервер. Данный продукт возник как доработанная версия другого HTTP-клиента от национального центра суперкомпьютерных приложений (NCSA).

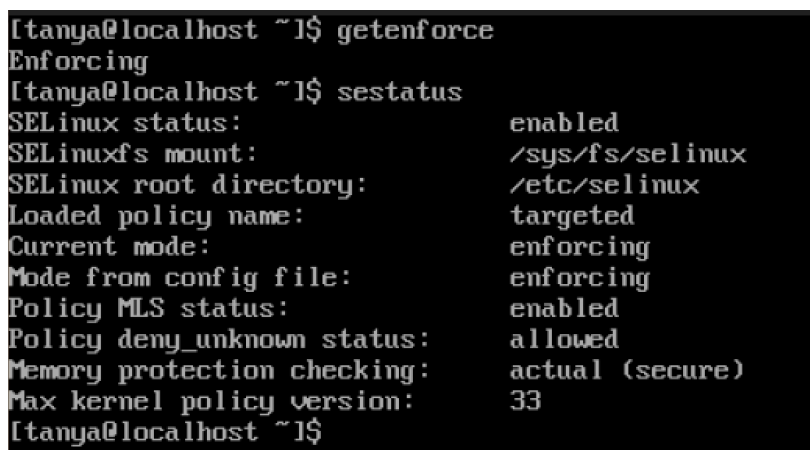
Для чего нужен Apache сервер:

- чтобы открывать динамические PHP-страницы,
- для распределения поступающей на сервер нагрузки,
- для обеспечения отказоустойчивости сервера,
- чтобы потренироваться в настройке сервера и запуске PHP-скриптов.

Apache является кроссплатформенным ПО и поддерживает такие операционные системы, как Linux, BSD, MacOS, Microsoft, BeOS и другие.

3 Выполнение лабораторной работы

Вошла в систему под своей учетной записью. Убедилась, что SELinux работает в режиме enforcing политики targeted с помощью команд `getenforce` и `sestatus` (рис. [fig:001]).



```
[tanya@localhost ~]$ getenforce
Enforcing
[tanya@localhost ~]$ sestatus
SELinux status:                enabled
SELinuxfs mount:                /sys/fs/selinux
SELinux root directory:         /etc/selinux
Loaded policy name:              targeted
Current mode:                    enforcing
Mode from config file:           enforcing
Policy MLS status:               enabled
Policy deny_unknown status:      allowed
Memory protection checking:      actual (secure)
Max kernel policy version:       33
[tanya@localhost ~]$
```

Рисунок 3.1: проверка режима работы SELinux

Запускаю сервер `apache`, далее обращаюсь с помощью браузера к веб-серверу, запущенному на компьютере, он работает, что видно из вывода команды `service httpd status` (рис. [fig:002]).


```

[tanya@localhost ~]$ sudo systemctl status httpd
■ httpd.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/httpd.service; enabled; preset: disabled)
   Active: active (running) since Wed 2025-08-26 13:20:37 MSK; 27s ago
     Invocation: bd3656d365ad48d0a67d0881c6b6bfd01
       Docs: man:httpd.service(8)
    Main PID: 32705 (httpd)
      Status: "Total requests: 0; Idle/Busy workers 100/0; Requests/sec: 0; Bytes served/sec: 0"
        Tasks: 177 (limit: 48828)
       Memory: 13.9M (peak: 16.2M)
          CPU: 2.208s
    CGroup: /system.slice/httpd.service
            └─32705 /usr/sbin/httpd -DFOREGROUND
              └─32706 /usr/sbin/httpd -DFOREGROUND
                └─32707 /usr/sbin/httpd -DFOREGROUND
                  └─32708 /usr/sbin/httpd -DFOREGROUND
                    └─32709 /usr/sbin/httpd -DFOREGROUND

```

Рисунок 3.2: Проверка работы Apache

С помощью команды `ps auxZ | grep httpd` нашла веб-сервер Apache в списке процессов. Его контекст безопасности - `httpd_t` (рис. [fig:003]).

```

[tanya@localhost ~]$ ps auxZ | grep httpd
system_u:system_r:httpd_t:s0 root 32705 0.0 0.1 18848 11044 ?
Ss 13:20 0:00 /usr/sbin/httpd -DFOREGROUND
system_u:system_r:httpd_t:s0 apache 32706 0.0 0.0 18600 5392 ?
S 13:20 0:00 /usr/sbin/httpd -DFOREGROUND
system_u:system_r:httpd_t:s0 apache 32707 0.5 0.1 2354284 8220 ?
Sl 13:20 0:01 /usr/sbin/httpd -DFOREGROUND
system_u:system_r:httpd_t:s0 apache 32708 0.4 0.1 2157612 7904 ?
Sl 13:20 0:00 /usr/sbin/httpd -DFOREGROUND
system_u:system_r:httpd_t:s0 apache 32709 0.3 0.1 2223148 7928 ?
Sl 13:20 0:00 /usr/sbin/httpd -DFOREGROUND
unconfined_u:unconfined_r:unconfined_t:s0-s0:c0.c1023 tanya 32899 0.0 0.0 22770
4 2228 tty1 S+ 13:23 0:00 grep --color=auto httpd

```

Рисунок 3.3: Контекст безопасности Apache

Просмотрела текущее состояние переключателей SELinux для Apache с помощью команды `sestatus -bigrep httpd` (рис. [fig:004]).

```

SELinux status:                enabled
SELinuxfs mount:              /sys/fs/selinux
SELinux root directory:      /etc/selinux
Loaded policy name:           targeted
Current mode:                 enforcing
Mode from config file:       enforcing
Policy MLS status:           enabled
Policy deny_unknown status:   allowed
Memory protection checking:   actual (secure)
Max kernel policy version:    33

Policy booleans:
abrt_anon_write                off
abrt_handle_event              on
abrt_upload_watch_anon_write   on
auditadm_exec_content          on
authlogin_nsswitch_use_ldap    off
authlogin_radius               off
authlogin_yubikey              off
cdrecord_read_content           off
cluster_can_network_connect    off
cluster_manage_all_files       off
cluster_use_execmem            off
colord_use_nfs                 off

```

Рисунок 3.4: Состояние переключателей SELinux

Просмотрела статистику по политике с помощью команды `seinfo`. Множество пользователей - 8, ролей - 39, типов - 5135. (рис. [fig:005]).

```

Statistics for policy file: /sys/fs/selinux/policy
Policy Version:            33 (MLS enabled)
Target Policy:              selinux
Handle unknown classes:    allow
Classes:                   133   Permissions:           458
Sensitivities:              1   Categories:           1024
Types:                     4660  Attributes:            245
Users:                      8    Roles:                 14
Booleans:                   314   Cond. Expr.:          339
Allow:                      57336 Neverallow:             0
Auditallow:                 149   Dontaudit:            7768
Type_trans:                 228528 Type_change:            74
Type_member:                 32   Range_trans:          3890
Role allow:                  36   Role_trans:           317
Constraints:                 70   Validatetrans:         0
MLS Constrains:             72   MLS Val. Tran:         0
Permissives:                 21   Polcap:                6
Defaults:                    7    Typebounds:            0
Allowxperm:                  0    Neverallowxperm:       0
Auditallowxperm:            0    Dontauditxperm:        0
Ibendportcon:               0    Ibpkeycon:             0
Initial SIDs:                27   Fs_use:                35
Genfscon:                   112   Portcon:               667
Netifcon:                    0    Nodecon:               0

```

Рисунок 3.5: Статистика по политике

Типы поддиректорий, находящихся в директории `/var/www`, с помощью команды `ls -lZ /var/www` следующие: владелец - root, права на изменения только у

владельца. Файлов в директории нет (рис. [fig:006]).

```
[tanya@localhost ~]$ ls -lZ /var/www
total 0
drwxr-xr-x. 2 root root system_u:object_r:httpd_sys_script_exec_t:s0 6 Jul  9 03
:00 cgi-bin
drwxr-xr-x. 2 root root system_u:object_r:httpd_sys_content_t:s0  6 Jul  9 03
:00 html
```

Рисунок 3.6: Типы поддиректорий

В директории /var/www/html нет файлов. (рис. [fig:007]).

```
[tanya@localhost ~]$ ls -lZ /var/www/html
total 0
[tanya@localhost ~]$
```

Рисунок 3.7: Типы файлов

Создать файл может только суперпользователь, поэтому от его имени создаем файл touch.html со следующим содержанием:

```
<html>
<body>test</body>
</html>
```

Листинг 1 (рис. [fig:008]).

```
[tanya@localhost ~]$ sudo touch /var/www/html/test.html
[sudo] password for tanya:
```

Рисунок 3.8: Создание файла

Проверяю контекст созданного файла. По умолчанию это httpd_sys_content_t (рис. [fig:009]).

```
[tanya@localhost ~]$ ls -lZ /var/www/html
total 4
-rw-r--r--. 1 root root unconfined_u:object_r:httpd_sys_content_t:s0 33 Aug 26 1
5:19 test.html
[tanya@localhost ~]$
```

Рисунок 3.9: Контекст файла

Обращаюсь к файлу через веб-сервер, введя в браузере адрес `http://127.0.0.1/test.html`.
Файл был успешно отображён (рис. [fig:010]).

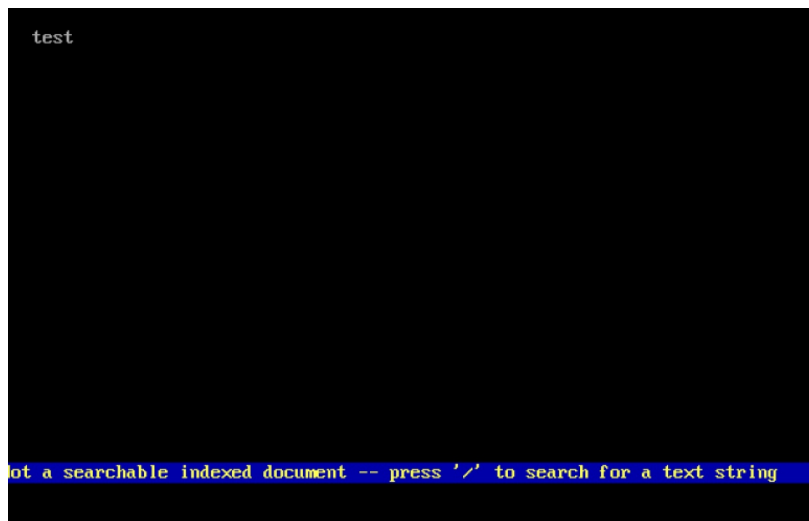


Рисунок 3.10: Отображение файла

Изучила справку `man httpd_selinux`. Рассмотрим полученный контекст детально. Так как по умолчанию пользователи CentOS являются свободными от типа (`unconfined` в переводе с англ. означает свободный), созданному нами файлу `test.html` был сопоставлен SELinux, пользователь `unconfined_u`. Это первая часть контекста. Далее политика ролевого разделения доступа RBAC используется процессами, но не файлами, поэтому роли не имеют никакого значения для файлов. Роль `object_r` используется по умолчанию для файлов на «постоянных» носителях и на сетевых файловых системах. (В директории `/ргос` файлы, относящиеся к процессам, могут иметь роль `system_r`. Если активна политика MLS, то могут использоваться и другие роли, например, `secadm_r`. Данный случай мы рассматривать не будем, как и предназначение `:s0`). Тип `httpd_sys_content_t` позволяет процессу `httpd` получить доступ к файлу. Благодаря наличию последнего типа мы получили доступ к файлу при обращении к нему через браузер. (рис. [fig:011]).

```

HTTPD(8)                                httpd                                HTTPD(8)

NAME
    httpd - Apache Hypertext Transfer Protocol Server

SYNOPSIS
    httpd [ -d serverroot ] [ -f config ] [ -C directive ] [ -c directive ]
    [ -D parameter ] [ -e level ] [ -E file ] [ -k start|restart|grace-
    full|stop|graceful-stop ] [ -h ] [ -l ] [ -L ] [ -S ] [ -t ] [ -v ] [ -U
    ] [ -X ] [ -M ] [ -T ]

    On Windows systems, the following additional arguments are available:

    httpd [ -k install|config|uninstall ] [ -n name ] [ -w ]

SUMMARY
    httpd is the Apache HyperText Transfer Protocol (HTTP) server program.
    It is designed to be run as a standalone daemon process. When used like
    this it will create a pool of child processes or threads to handle re-
    quests.

    In general, httpd should not be invoked directly, but rather should be
    invoked via apachectl on Unix-based systems or as a service on Windows
    NT, 2000 and XP and as a console application on Windows 9x and ME.

Manual page httpd(8) line 1 (press h for help or q to quit)

```

Рисунок 3.11: Изучение справки по команде

Изменяю контекст файла `/var/www/html/test.html` `chcon httpd_sys_content_t` на любой другой, к которому процесс `httpd` не должен иметь доступа, например, `samba_share_t`: `chcon -t samba_share_t /var/www/html/test.html`
`ls -Z /var/www/html/test.html` Контекст действительно поменялся (рис. [fig:012]).

```

[tanya@localhost ~]$ ls -Z /var/www/html/test.html
unconfined_u:object_r:httpd_sys_content_t:s0 /var/www/html/test.html
[tanya@localhost ~]$ sudo chcon -t samba_share_t /var/www/html/test.html
[sudo] password for tanya:
[tanya@localhost ~]$ ls -lZ /var/www/html
total 4
-rw-r--r--. 1 root root unconfined_u:object_r:samba_share_t:s0 33 Aug 26 15:19 test.html

```

Рисунок 3.12: Изменение контекста

При попытке отображения файла в браузере получаем сообщение об ошибке (рис. [fig:013]).



Рисунок 3.13: Отображение файла

файл не был отображён, хотя права доступа позволяют читать этот файл любому пользователю, потому что установлен контекст, к которому процесс httpd не должен иметь доступа.

Просматриваю log-файлы веб-сервера Apache и системный лог-файл: `tail /var/log/messages`. Если в системе окажутся запущенными процессы `setroubleshootd` и `audtd`, то вы также сможете увидеть ошибки, аналогичные указанным выше, в файле `/var/log/audit/audit.log`. (рис. [fig:014]).

```
[tanya@localhost ~]$ ls -l /var/www/html/test.html
-rw-r--r--. 1 root root 33 Aug 26 15:19 /var/www/html/test.html
[tanya@localhost ~]$ tail /var/log/messages
tail: cannot open '/var/log/messages' for reading: Permission denied
[tanya@localhost ~]$ sudo tail /var/log/messages
Aug 26 13:36:54 localhost systemd[1]: run-p33120-i33121.service: Deactivated successfully.
Aug 26 14:51:29 localhost systemd[1]: Starting dnf-makecache.service - dnf makecache...
Aug 26 14:51:30 localhost dnf[34567]: Metadata timer caching disabled when running on a battery.
Aug 26 14:51:30 localhost systemd[1]: dnf-makecache.service: Deactivated successfully.
Aug 26 14:51:30 localhost systemd[1]: Finished dnf-makecache.service - dnf makecache.
Aug 26 15:07:58 localhost systemd[1]: Started run-p34660-i34661.service - [systemd-run] /usr/bin/systemctl start man-db-cache-update.
Aug 26 15:07:59 localhost systemd[1]: Starting man-db-cache-update.service...
Aug 26 15:08:00 localhost systemd[1]: man-db-cache-update.service: Deactivated successfully.
Aug 26 15:08:00 localhost systemd[1]: Finished man-db-cache-update.service.
Aug 26 15:08:00 localhost systemd[1]: run-p34660-i34661.service: Deactivated successfully.
```

Рисунок 3.14: Попытка прочесть лог-файл

Чтобы запустить веб-сервер Apache на прослушивание TCP-порта 81 (а не 80, как

рекомендует IANA и прописано в /etc/services) открываю файл /etc/httpd/httpd.conf для изменения. (рис. [fig:015]).

Left	File	Command	Options
<-	/etc/httpd/conf		.[^]>
.n	Name	Size	Modify time
/..		UP--DIR	Aug 26 13:19
httpd.conf		12005	Jul 9 03:00
magic		13430	Jul 9 03:00
httpd.conf			
32G / 35G (91%)			
Hint: Leap to frequently used directories			

Рисунок 3.15: Изменение файла

Нахожу строчку Listen 80 и заменяю её на Listen 81. (рис. [fig:016]).

```

httpd.conf  [-M--]  9 L:[ 30*17  47/359] *(2025/12005b) 0010 0x000 [-*]IX
# Mutex directive, if file-based mutexes are used.  If you wish to share the
# same ServerRoot for multiple httpd daemons, you will need to change at
# least PidFile.
#
ServerRoot "/etc/httpd"
#
# Listen: Allows you to bind Apache to specific IP addresses and/or
# ports, instead of the default.  See also the <VirtualHost>
# directive.
#
# Change this to Listen on a specific IP address, but note that if
# httpd.service is enabled to run at boot time, the address may not be
# available when the service starts.  See the httpd.service(8) man
# page for more information.
#
Listen 12.34.56.78:80
Listen 81_
#
# Dynamic Shared Object (DSO) Support
#
# To be able to use the functionality of a module which was built as a DSO you

```

Рисунок 3.16: Изменение порта

Выполняю перезапуск веб-сервера Apache. Произошёл сбой, потому что порт 80 для локальной сети, а 81 нет (рис. [fig:017]).

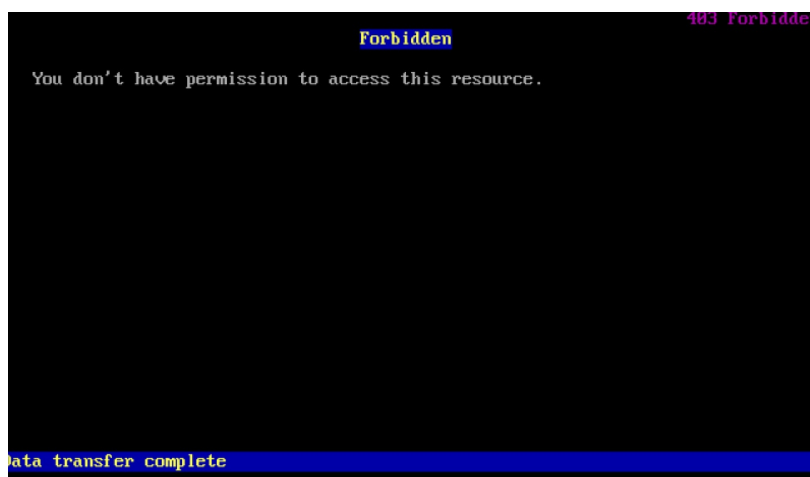


Рисунок 3.17: Попытка прослушивания другого порта

Проанализируйте лог-файлы: `tail -n1 /var/log/messages` (рис. [fig:018]).

Посмотрите файлы `/var/log/http/error_log`, `/var/log/http/access_log` и `/var/log/audit/audit.log` и выясните, в каких файлах появились записи. Запись появилась в файле `error_log`.

Выполняю команду `semanage port -a -t http_port_t -p tcp 81`
После этого проверяю список портов командой `semanage port -l | grep http_port_t` Порт 81 появился в списке.

Перезапускаю сервер Apache (рис. [fig:018]).

```
[tanya@localhost ~]$ sudo semanage port -a -t http_port_t -p tcp 81
Port tcp/81 already defined, modifying instead
[13848.605914] SELinux: Converting 492 SID table entries...
[13848.623638] SELinux: policy capability network_peer_controls=1
[13848.624998] SELinux: policy capability open_perms=1
[13848.625658] SELinux: policy capability extended_socket_class=1
[13848.626439] SELinux: policy capability always_check_network=0
[13848.627196] SELinux: policy capability cgroup_seclabel=1
[13848.628427] SELinux: policy capability mmp_nosuid_transition=1
[13848.629093] SELinux: policy capability genfs_seclabel_symlinks=1
[13848.629741] SELinux: policy capability ioctl_skip_cloexec=0
[13848.631369] SELinux: policy capability userspace_initial_context=0
[tanya@localhost ~]$ sudo systemctl restart httpd
chcon: failed to change context of '/var/www/html/test.html' to 'unconfined_u:object_r:httpd_sys_content_t:s0': Operation not permitted
[tanya@localhost ~]$ sudo chcon -t httpd_sys_content_t /var/www/html/test.html
```

Рисунок 3.18: Перезапуск сервера

Теперь он работает, ведь мы внесли порт 81 в список портов `httpd_port_t` (рис. [fig:019]).



Рисунок 3.19: Проверка сервера

Возвращаю в файле `/etc/httpd/httpd.conf` порт 80, вместо 81. Проверяю, что порт 81 удален, это правда. (рис. [fig:020]).

```

[tanya@localhost ~]$ semanage port -d -t http_port_t -p tcp 81
ValueError: SELinux policy is not managed or store cannot be accessed.
[tanya@localhost ~]$ sudo semanage port -d -t http_port_t -p tcp 81
ValueError: Port tcp/81 is defined in policy, cannot be deleted

```

Рисунок 3.20: Проверка порта 81

Далее удаляю файл test.html, проверяю, что он удален(рис. [fig:021]).

```

[tanya@localhost ~]$ semanage port -d -t http_port_t -p tcp 81
ValueError: SELinux policy is not managed or store cannot be accessed.
[tanya@localhost ~]$ sudo semanage port -d -t http_port_t -p tcp 81
ValueError: Port tcp/81 is defined in policy, cannot be deleted
[tanya@localhost ~]$ sudo rm /var/www/html/test.html
[tanya@localhost ~]$ sudo -lZ /var/www/html/test.html
sudo: invalid option -- 'Z'
usage: sudo -h | -K | -k | -U
usage: sudo -v [-ABkMnS] [-g group] [-h host] [-p prompt] [-u user]
usage: sudo -l [-ABkMnS] [-g group] [-h host] [-p prompt] [-U user]
        [-u user] [command [arg ...]]
usage: sudo [-ABbEHkMnPS] [-r role] [-t type] [-C num] [-D directory]
        [-g group] [-h host] [-p prompt] [-R directory] [-T timeout]
        [-u user] [VAR=value] [-i | -s] [command [arg ...]]
usage: sudo -e [-ABkMnS] [-r role] [-t type] [-C num] [-D directory]
        [-g group] [-h host] [-p prompt] [-R directory] [-T timeout]
        [-u user] file ...
[tanya@localhost ~]$ sudo ls -lZ /var/www/html/test.html
ls: cannot access '/var/www/html/test.html': No such file or directory
[tanya@localhost ~]$ sudo ls -lZ /var/www/html
total 0

```

Рисунок 3.21: Удаление файла

4 Выводы

В ходе выполнения данной лабораторной работы, я развила навыки администрирования ОС Linux, получила первого практического знакомства с технологией SELinux, а также проверила работу на практике совместно с веб-сервером Apache.

Список литературы

[0] Методические рекомендации курса